

Dung (Dzung) Than
BIOL 3999
April 30, 2024

Investigation of Intron Signal in RNA-Seq

Abstract:

Genes consist of exons separated by introns. Exons have naturally been the focus of attention, however, the advent of RNA-Seq has revealed a hitherto unseen world of intron signal that demands explanation. Here we focus on the basic question about what extent of observed intron signal in RNA-Seq is due to immature pre-RNA that still contain the introns connected to the exons, and to what extent (if any) is the signal due to free floating introns which have been excised but have not been degraded. Ultimately, we are interested to know if any such molecules are functional, and how we might infer that. Utilizing tools previously developed in the ITMAT Bioinformatics Laboratory, and adding additional tools, we have compiled a list of candidate genes for follow-up. We have also performed a tissue survey to assess whether the nature of free-floating introns varies by tissue.

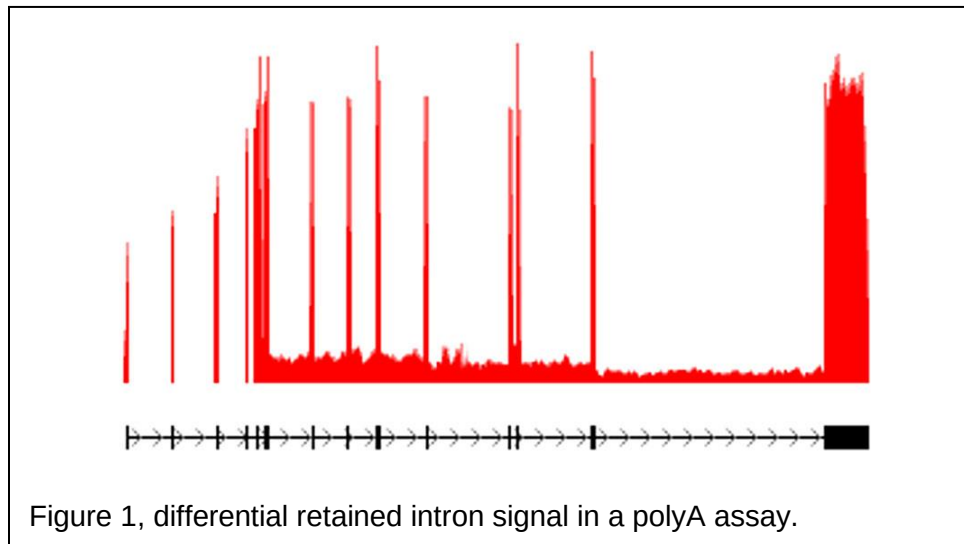
Introduction:

RNA sequencing (RNA-Seq) has profoundly influenced our understanding of transcriptional landscapes by allowing detailed quantification of RNA transcripts. However, the focus of most RNA-Seq analyses has predominantly been on coding regions of mRNA and exons in general, notably overlooking introns. Introns are intervening sequences, excised from pre-mRNA soon after their transcription and thought to be rapidly degraded. This prevailing view, based on their rapid excision and degradation, suggests a limited biological role post-transcription (Berget et al., 1977).

Contrary to this traditional perspective, emerging evidence suggests that introns may have functions that extend beyond their canonical roles in RNA splicing. Research indicates that retained introns can influence mRNA export, stability, and localization, and may even participate in regulatory networks (Lejeune & Maquat, 2005; Braunschweig et al., 2014). Nonetheless, the question of whether intronic sequences are retained in order to play some sort of functional role remains largely unexplored. We endeavored to investigate the feasibility of identifying free floating intron molecules from RNA-Seq. Typically, millions of short sequence reads are generated, which are then aligned to the genome and represented by a depth-of-coverage plot. Although such plots provide a good measure of the amount of retained intron signal, they cannot give a visual assessment of how much, if any, of the intron signal is coming from introns that are no longer connected to their adjacent exons.

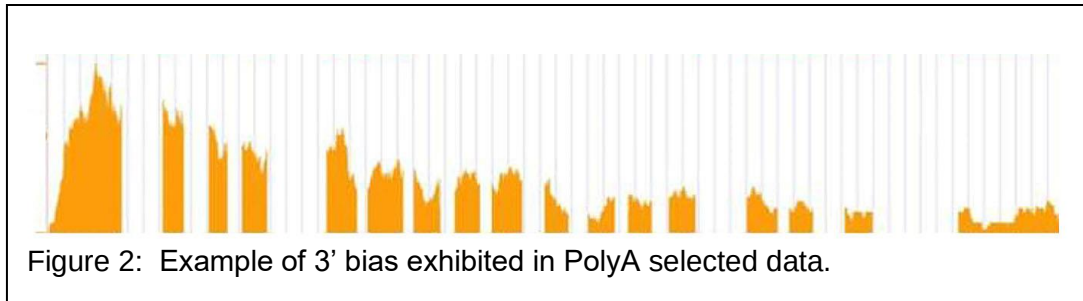
The standard practice in RNA-Seq involves polyadenylation (polyA) selection to enrich for mature mRNA, effectively sidelining non-polyadenylated RNA forms, including most intronic RNA (Mortazavi et al., 2008). This method aligns with the primary interest of most researchers in examining the exon signals from mRNA. Additionally, approximately half of non-coding RNAs

also contain polyA tails. Conversely, intronic sequences, which are generally excised and degraded post-transcription, are unlikely to be polyadenylated. Consequently, polyA-selected assays are presumed to largely, if not completely, lack signals from free-floating introns. Nevertheless, these assays frequently exhibit substantial intron signals, ranging from low levels typically associated with precursor mRNA (pre-mRNA) to unexpectedly high levels of retained introns. Figure 1 shows a data from a polyA selected assay showing some highly retained introns while others are apparently completely eliminated. This cannot easily be explained entirely by the normal processing of preRNA into its mature state.

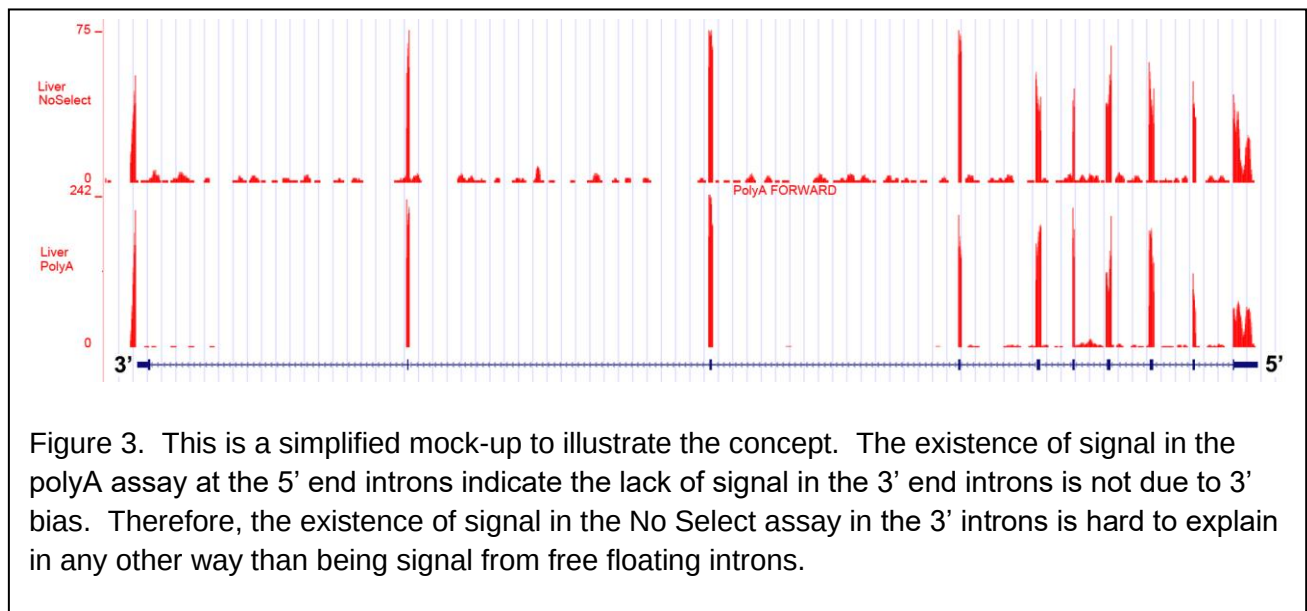


It is expected that in a polyA assay, little if any detected intron signal originates from free-floating introns. Rather signal should be entirely from the retained introns. This apparent deficiency actually serves as a highly informative control, if compared to a different type of assay designed to capture all sequences, not just polyadenylated ones. In pursuit of this, the ITMAT Bioinformatics Laboratory has generated assays using the same samples in both polyA-selected and non-ribosomal-depleted protocols. To ensure comparability in sequencing depth on non-ribosomal RNA, as total RNA is 80-90% ribosomal, assays without ribosomal RNA depletion were sequenced with ten times as many reads. This experimental design ensures comparability between assays of the same sample, where one set undergoes polyA selection and the other does not.

If we can identify the presence of intron signal exclusively in the "no select" assay, which should be absent in the polyA-selected assay, that should indicate the presence of free-floating intron molecules. However, one must account for technical artifacts associated with polyA sequencing, which can otherwise severely confound the results. Specifically, since the polyA tail is attached to the 3' end of RNA molecules, breaks occurring in the RNA—whether from native degradation or during library preparation — results in sequencing predominantly the 3' ends of molecules. Consequently, polyA-selected data exhibit a 3' bias, where signal intensity decreases from the 3' end towards the 5' end of the RNA. See Figure 2.



Thus, an intron located at the 5' end of an RNA molecule may manifest a lower signal in the polyA assay compared to the "no select" assay simply because of this 3' bias – a discrepancy not therefore indicative of free-floating introns. We control for this bias by comparing consecutive introns within the same transcript. If an intron demonstrates sufficiently higher signal in the "no select" assay compared to the polyA, and the same intron simultaneously has *lower* signal in the polyA assay than another intron positioned further along the 3' direction of the transcript, then it can be considered a candidate for representing free-floating intron signal. This is illustrated in Figure 3.



This study aims to fill a significant gap in current transcriptomic analysis methods by systematically examining the phenomenon of intron retention. We have demonstrated this using *Mus musculus* mm10 genome, aligned to GENCODE v23 annotation. By clarifying the extent and nature of intron retention, we aim to expand the understanding of intron functionality in the post-transcriptional regulation of gene expression.

Overview:

This research is built on work by previous students in Dr. Grant's Lab, which has resulted in a toolkit of scripts for data extraction. The first script is used to identify unambiguous introns in the GENCODE genome assembly. This provides a canonical set of introns that can be used to investigate issues without the results being confused by overlapping genes or alternate splicing, for example cassette exons, or exons with varying 3' or 5' splice sites across different isoforms. An intron is *unambiguous* if it shares the left and right splice junctions with all overlapping introns in any other annotated isoform or overlapping gene. This script identifies the unambiguous introns from a given set of gene annotation. The second script is used to count the numbers of reads crossing intron-exon junctions and the number of reads which splice at the intron-exon junctions, for all unambiguous introns determined in the previous step. The last script was used to calculate the average depth of coverage for each unambiguous intron.

The scripts have previously been applied to several RNA-Seq data sets generated in-house. These datasets include ones which were prepared without depleting rRNA, and the sequencing depth was enhanced tenfold, resulting in approximately 300 million reads. This adjustment aimed to achieve a signal depth on non-ribosomal RNA that is equivalent to that obtained under conditions where rRNA depletion occurred, and the samples were sequenced at a standard depth of around 30 million reads. In order to utilize the strategy described above to identify candidates for free floating intron signal, samples from mouse heart, lung, and liver were prepared using both polyA selection and another library preparation which involved no rRNA depletion at all, or a different strategy for rRNA depletion (Ribo-Zero) which retains all sequences, not just those that are polyadenylated.

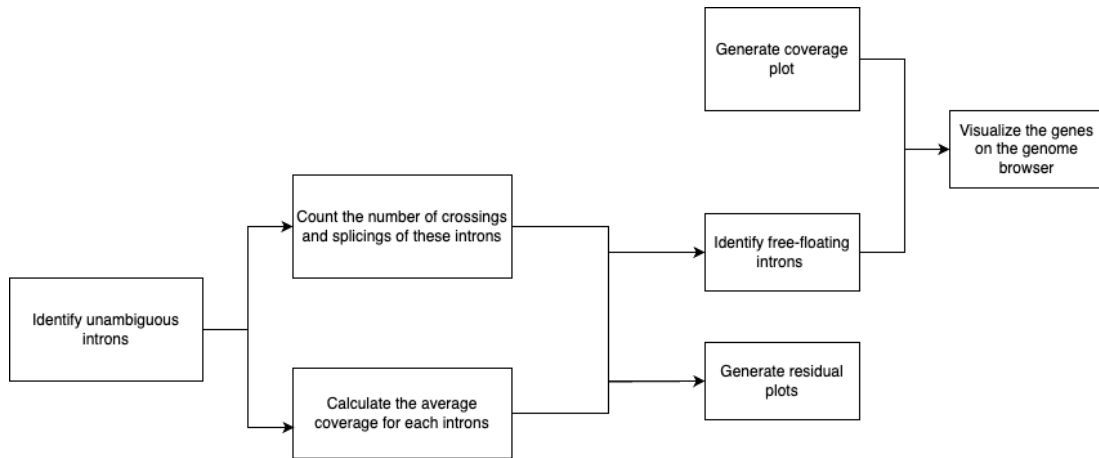
Building upon prior work, I endeavored to apply the developed code to multiple data sets, including public data. Having identified several coding issues in all of the scripts, however, it became necessary to first rewrite them all from scratch. The code is now fully vetted, vastly more efficient and well documented.

Comparing counts of spliced and unspliced intron signals and overall intron expression of all the exon-intron junctions in the in-house datasets, informative plots are generated, to assess the degree of evidence for free floating introns. In particular, for each intron/exon junction, the average coverage (X -axis) is plotted against the number of reads aligning across the exon-intron junction (thereby indicating a retained intron). The higher the average coverage is as compared to the number of crossing reads, the stronger the evidence for free floating introns. Any point below the diagonal is considered poor evidence. We calculate a summary statistic to compare the overall effect across samples (e.g. to compare different tissues). We also identify and characterize a list of candidate introns displaying unusual behavior, for further validation.

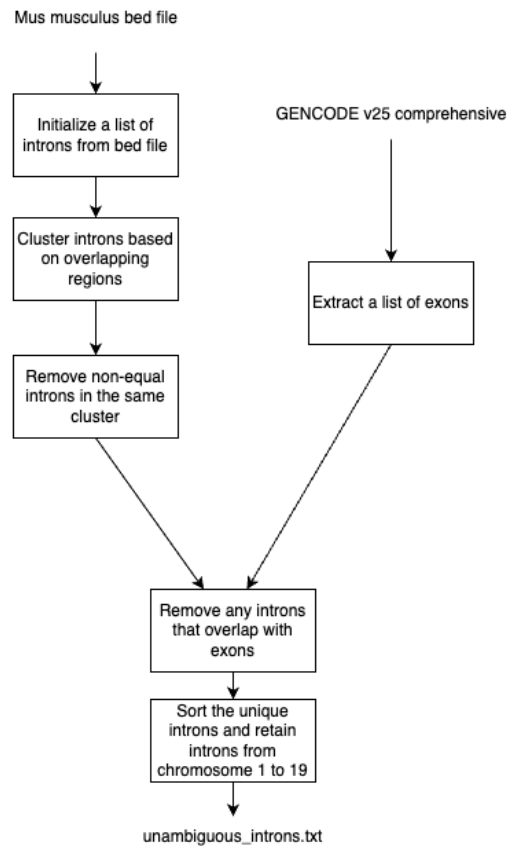
A tissue survey was performed by tapping into the extensive publicly available sequence data in the Gene Expression Omnibus (GEO). These complementary analyses of in-house and public data will validate findings and contribute to determining the nature of intron signal. This investigation, though largely hypothesis-generating, promises to shed light on some of the many mysteries of intron signals within RNA-Seq results.

Methods:

Overview:



Identifying unambiguous introns:



Counting the number of crossings and splicing:

Due to errors in previous scripts, I wrote a new python script to count the number of crossings and splicings of each junction. This script takes in as inputs the intron list file, the SAM

file corresponding to each sample, the output files for crossings information, and the output file for splicings information. This script is provided in the Appendix.

To prepare the data, the intron coordinates are loaded from a specified input file, formatted to include chromosome, start, end, strand, and identifier. The script standardizes chromosome identifiers by removing 'chr' prefixes and converting 'X' and 'Y' to numerical values. It organizes intron data into structured arrays, segregating by chromosome and strand, and initializes counters for tracking junctions.

The script processes the SAM file line by line. Each line, representing a sequencing read, is passed to the prepare function, which parses and formats necessary data from the SAM entry, including chromosome, start position, and CIGAR string. This function ensures data compatibility by standardizing chromosome identifiers and filtering entries based on predefined criteria.

For each formatted read, the doWork function is invoked. This function analyzes the read's alignment to detect whether it crosses or aligns with annotated intron junctions based on the CIGAR string. It examines both the start and end of introns, comparing alignment positions with intron junction positions to count and record junction crossings and potential splicing events.

Following the analysis of paired-end reads, the fixDups function is used to address potential double-counting of junction crossings and splicings due to read overlaps. This function adjusts recorded counts by cross-referencing detected events and correcting over-counted entries, ensuring the accuracy of the data.

After processing all reads, the script outputs the counts of junction crossings and splicings to the designated files. Each output entry includes details such as chromosome, junction position, strand orientation, and the count of events, providing a comprehensive dataset for further analysis.

Calculate the average coverage of each intron:

Due to issues in previous scripts, I also developed a new Python script to compute the average coverage for each intron. This script accepts as inputs the intron list file, the forward strand coverage file, the reverse strand coverage file, and the output file for the final coverage information. This script is included in the Appendix.

To prepare the data, intron coordinates are loaded from a specified input file, formatted to include chromosome, start, end, strand, and identifier. Chromosome identifiers are standardized by removing 'chr' prefixes and converting 'X' and 'Y' to numeric values. The script organizes intron data into structured arrays, segregating by chromosome and strand, and initializes counters to track coverage.

The script processes the forward and reverse strand coverage files separately. It first checks for a header in the coverage files and skips it if present. For each chromosome, the script aligns the intron data with the corresponding coverage data. It reads the intron list and coverage data line by line, parsing and formatting essential information, including chromosome, start, end, and coverage values. This ensures data consistency by standardizing chromosome identifiers and filtering entries based on specific criteria.

For each intron, the do_work function is called. This function calculates the total coverage within the intron boundaries by comparing the alignment positions of the coverage data with the intron positions. It examines both the start and end of introns, summing the coverage values that

overlap with the intron regions. The average coverage for each intron is then computed and recorded.

After processing both forward and reverse strand coverage files, the `merge_files` function combines the results into a single dataset. This function reads the temporary files generated during processing, eliminates duplicates, and merges the data into a unified output. The `sort_and_finalize` function sorts the merged data by chromosome and intron positions, converting numeric chromosome identifiers back to 'X' and 'Y' where applicable. The final sorted data is written to the output file.

After processing all data, the script outputs the average coverage for each intron to the designated file. Each output entry includes details such as chromosome, intron start and end positions, strand orientation, total coverage, intron length, and average coverage, providing a comprehensive dataset for further analysis.

Identifying free-floating introns:

We will identify free-floating RNAs using three criteria below:

- Low Signal in PolyA (3' Most Intron): The signal should be exceptionally low, ideally near zero, in the 3' most intron when measured with a polyA assay. This criterion is essential to minimize the impact of the 3' bias inherent in polyA assays, which typically reduces signal intensity toward the 5' end of transcripts.
- Higher Signal in the Subsequent Intron (PolyA): There should be an increase in signal in the intron immediately following the 3' most intron within the polyA assay. This increase indicates that the low signal observed in the 3' most intron is not merely a result of 3' bias. If it were due to 3' bias, the signal in the subsequent intron would be lower. Additionally, the presence of signal in this subsequent intron suggests it is still connected to exons, raising the possibility that it is the 3' most intron that is actually free-floating.
- Non-Low Signal in NoSelect (3' Most Intron): The intron signal in the 3' most intron should not be low in the NoSelect assay, and is preferably high. This indicates that the intron is actively transcribed, supporting the hypothesis that the low signal in the polyA assay is not an artifact but potentially indicative of a free-floating intron.

Incorporating these criteria, our script executes the following steps. Initially, we extracted all consecutive intron pairs from the GENCODE annotation file. Each pair was then scrutinized for an increase in intronic coverage from the first intron to the subsequent one in the polyA-selected RNA sequencing assay. Further analysis involved accessing the 'avecov' dataset, which provided coverage values for each intron identified in the initial step. The crucial criterion for an intron pair to be classified as 'interesting' involved a specific signal pattern across assays: in the polyA assay, the coverage of the second intron should be significantly higher than that of the first (with a predefined ratio of 0.3), whereas in the 'noSelect' assay, the first intron's signal should surpass that of the second.

Conduct tissue survey through public data from GEO:

To conduct a comprehensive tissue survey, we identified wight additional publicly available datasets from the Gene Expression Omnibus (GEO) database. We specifically targeted tissues in wildtype *Mus musculus* cells to avoid any modifications in gene expressions. To identify the potential presence of free-floating introns, we focused on datasets that were sequenced without using the polyA selection method. Additionally, we prioritized paired-end data to ensure accurate alignment and quantification of intronic regions. We selected eight tissues for this analysis: embryonic stem cells, pancreatic islet cells, lung, kidney, hippocampus, pituitary, quadriceps, and colon.

From the GEO datasets, we extracted their respective fastQ files. Using the STAR alignment tool, we generated the coverage files and SAM files for each tissue. These generated coverage files, along with the SAM files, were then processed through the same data pipeline as our in-house *Mus musculus* liver data to count the number of crossings and splicings and to calculate the intron average coverage.

Results:

The strategy for determining candidate free-floating introns for validation as described in Methods resulted in a short list of 59 introns (see Appendix for full list). These were subsequently evaluated by hand to identify the most promising ones. About half were ruled out as false positives for various reasons. A short list of 20 were determined to have fairly strong evidence. Figures 4 through 6 show examples.

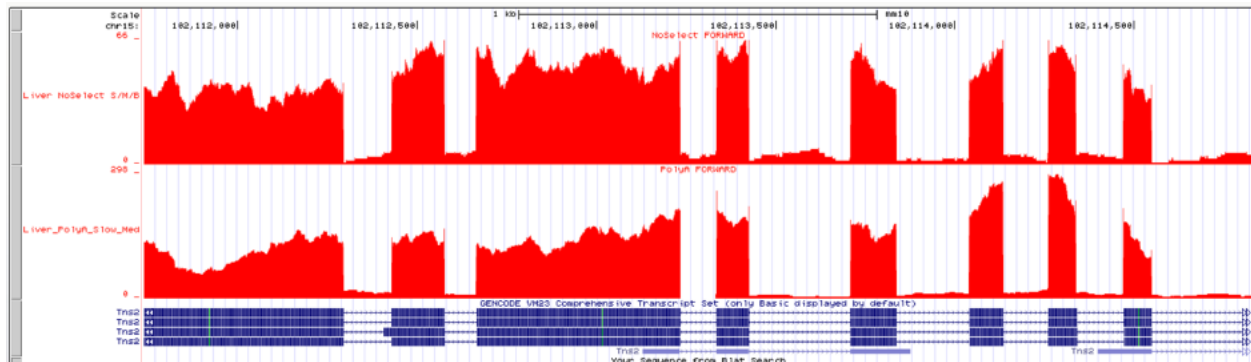


Figure 4: Genome Browser snapshot of chr15:102,111,739-102,114,828

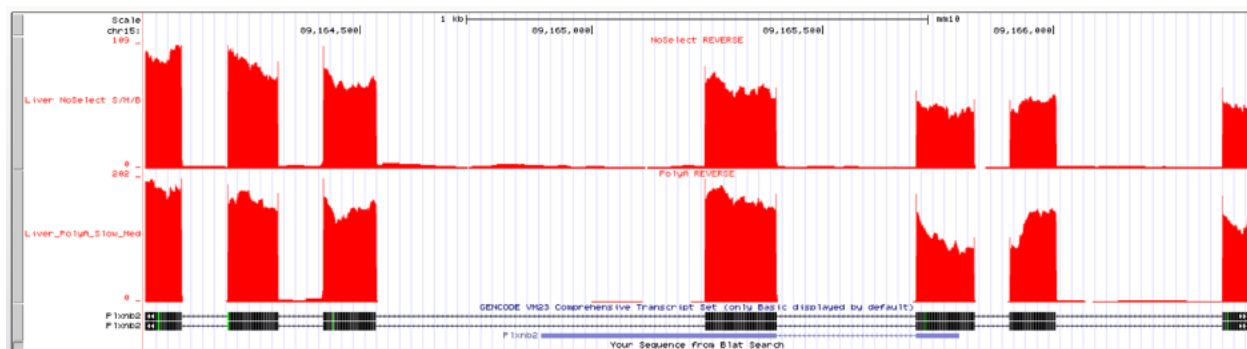


Figure 5: Genome Browser snapshot of chr15:89,164,035-89,166,42

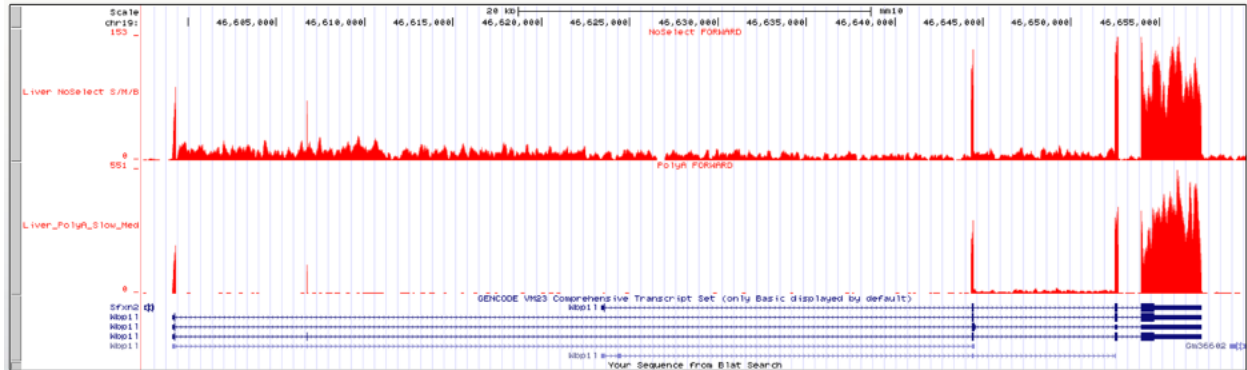


Figure 6: Genome Browser snapshot of chr19:46,597,468-46,659,898

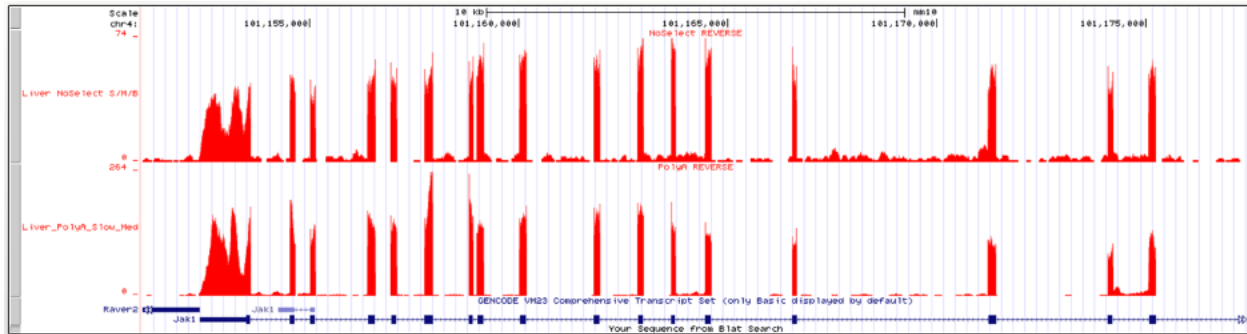


Figure 7: Genome Browser snapshot of chr4:101,151,011-101,177,439

Tissue Survey: To compare the extent of free floating introns between tissues or experimental conditions, we developed a summary statistic which is the sum over all exon/intron junctions of the average coverage minus the number of crossing reads. The sum being taken only over exon/intron junctions where this difference is positive. The rationale is that the more a particular intron exhibits free floating signal, then the greater the average coverage should be with respect to the number of crossing reads, since crossing reads are evidence of intron retention. The sum is then normalized by the read depth. We then compare this summary statistic across WT mouse tissues, see Figure 8. There appears to be quite a lot of variability by tissue, with Kidney, Brain and Quadriceps being the highest. The polyA sample was included as a negative control.

The

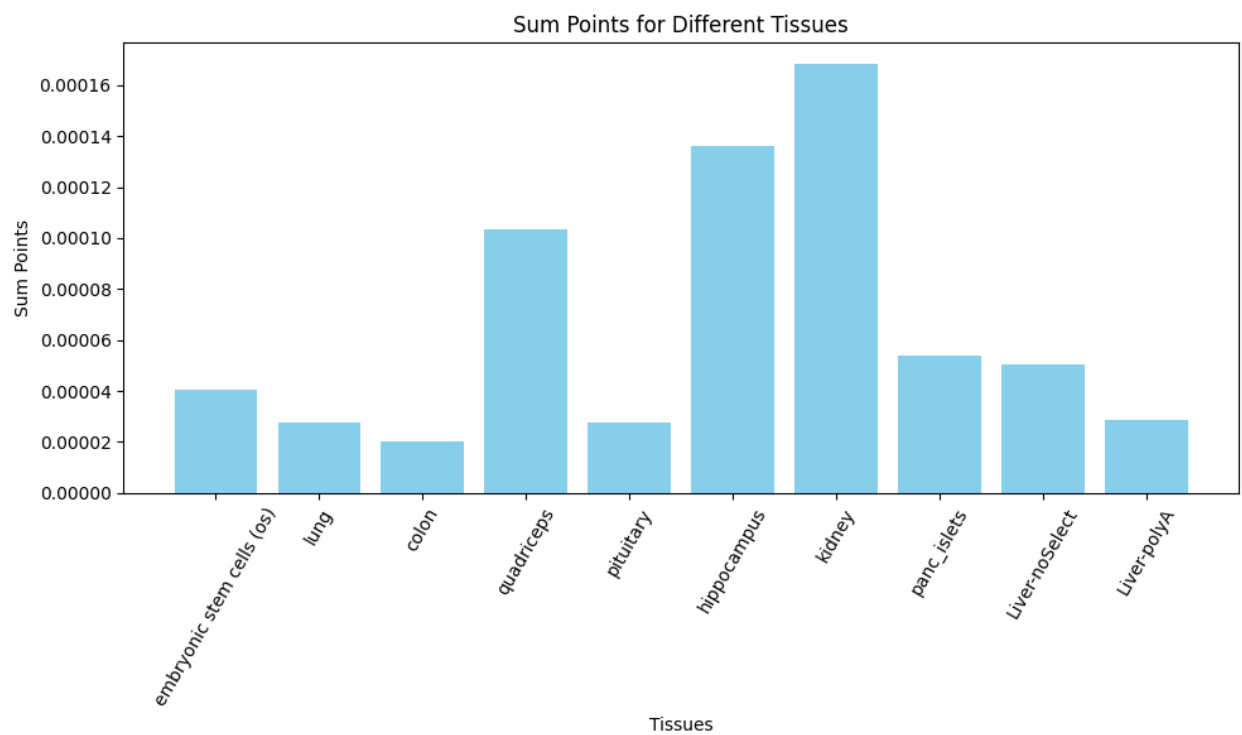


Figure 8: Sum of the Ratios of Residuals to Crossings, Normalized by Depth, in Each Tissue
For each tissue, we calculate the ratio of residuals to crossings of each junction, sum them up. We then normalized the sum points by the depth of each tissue.

Validating Experiment:

To validate the presence of the free-floating introns, we will use CAGE-seq. CAGE-seq (Cap Analysis of Gene Expression by sequencing) is an analytical technique utilized to quantify gene expression and pinpoint transcription start sites (TSS) within the genome. This process commences with the isolation of RNA, emphasizing mRNA molecules characterized by a 5' cap. These molecules are subsequently reverse-transcribed into cDNA using random primers. A distinctive feature of CAGE-seq involves the biotinylation of the 5' caps, facilitating the selective enrichment of capped cDNA. RNA strands that are either uncapped or not fully transcribed to cDNA are eliminated through RNase digestion. The biotinylated cDNA fragments are captured using streptavidin-coated beads and prepared for sequencing. Through this sequencing, CAGE-seq provides detailed insights into transcriptional activity, revealing both the abundance of transcripts and the precise locations of TSS ("CAGE-Seq," 2024).

Using CAGE-seq, we will analyze the sequencing data to identify the locations of TSS relative to known exon and intron boundaries. By determining whether a 5' end is intronic, we can infer whether the transcription is initiated within an intron, suggesting that the intron may be free-floating and not connected to an exon. Conversely, a 5' end located within an exonic region indicates that the intron is connected to an exon.

Discussions:

After running our analysis, we were able to identify four regions that suggests the existence of free-floating introns as illustrated in figure 4, 5, 6, and 7. In these figures, there are more signals in upstream introns compared to some downstream ones in polyA samples, suggesting that there is little to no 3' bias in this region. Looking at the same introns in the noSelect sample, we see that the downstream intron has just as much intron signal, if not more than, the introns upstream. Since we are able to rule out 3' bias in this region, this suggests the presence of free-floating introns in the intron downstream that was not captured in the polyA sample.

Residual plots for the polyA and noSelect samples reflect a somewhat similar trend in the relationship between AveCov and the number of crossings in each sample. In the polyA sample, the residuals are distributed across a board range of crossings values, from 0 to approximately 900, with the majority of data points concentrated below 400 crossings. Most residuals lie between -200 and 200, indication moderate variance. In the noSelect sample, the plot shows a more constrained range of crossings (up to about 220) and residuals mostly contained within -50 to 50. This suggests a tighter model fit compared to the polyA sample. For both samples, the residuals generally hover around zero line, but do not show a consistent pattern or trend, suggesting no major systematic biases. However, the closer clustering of points around the zero line also suggests lower variability in model performance compared to the polyA sample.

There are numerous directions for follow-up including expanding the tissue survey, developing a wet-bench validation assay and eventually performing functional assays.

Appendix:

Short list of Candidates:

```
chr6:90,346,305-90,346,635
chr5:31,305,640-31,309,519
chr15:96,990,409-97,063,128
chr4:101,142,355-101,271,954
chr8:70,331,102-70,353,466
chr13:55,160,923-55,161,864
chr17:35,160,688-35,162,666
chr10:77,053,212-77,054,231
chr11:4,030,231-4,052,304 ***
chr11:69,102,659-69,103,769
chr11:94,336,363-94,383,140
chr13:55,160,736-55,161,852
chr15:102,111,739-102,114,828 **
chr15:89,164,035-89,166,421 **
chr16:90,227,447-90,284,915 **
```

```

chr17:32,504,917-32,530,922
chr17:46,678,136-46,680,888
chr17:84,682,353-84,700,868
chr17:84,704,489-84,791,785 **
chr17:84,656,548-84,683,641 ***
chr18:74,273,096-74,274,074 ***
chr19:46,501,529-46,517,895 **
chr19:46,597,468-46,659,898 ***
chr19:46,572,914-46,598,237 **
chr19:6,275,314-6,301,016
chr19:8,914,221-8,915,740
chr1:55,079,074-55,088,469
chr1:6,207,097-6,279,996
chr1:74,731,392-74,738,192
chr2:104,718,778-104,743,493
chr2:143,946,732-143,953,451
chr2:25,278,240-25,286,039
chr2:26,410,462-26,445,732 **
chr3:121,758,766-121,823,565 **
chr4:101,151,011-101,177,439 ***
chr4:43,539,050-43,567,467 ***
chr4:58,797,497-58,841,296
chr5:115,111,077-115,112,277 ***
chr5:31,296,024-31,327,904 **
chr5:8,960,501-8,998,204 **
chr6:113,602,746-113,618,983
chr6:113,635,096-113,696,395 **
chr6:42,355,625-42,375,724
chr7:111,072,474-111,074,023
chr7:126,546,009-126,566,622
chr8:106,131,198-106,133,267 **
chr8:3,388,678-3,457,115
chr8:4,244,697-4,245,683
chr8:45,238,236-45,263,386 **
chr8:70,331,087-70,353,450
chr8:70,762,706-70,764,238
chr8:71,395,606-71,405,211
chr8:85,488,172-85,531,450 ***
chr9:15,282,606-15,302,655
chr9:21,966,253-21,993,590
chr9:35,210,705-35,217,088 **
chr9:77,747,458-77,796,762 *
chrX:73,714,293-73,741,625
chrX:73,786,629-73,792,754

```

Counting crossings and splicings code:

```

import sys
import re

# Check for the correct number of command-line argument
intron_file = sys.argv[1]
samfile = sys.argv[2]
crossings_outfile = sys.argv[3]
splicings_outfile = sys.argv[4]

overlap_cutoff = 8
junctions = {'SPOF': {}, 'SPOR': {}, 'EPOF': {}, 'EPOR': {}}
counts = {'F': [0]*22, 'R': [0]*22}

def prepare(line):
    # Strip newline characters and skip lines starting with '@'

```

```

line = line.strip()
if line.startswith('@'):
    return None

# Split the line on tabs
fields = line.split('\t')
bitflag = int(fields[1])
read_reverse_strand = (bitflag & 16) != 0
first_in_pair = (bitflag & 64) != 0
NH = int(re.search(r'NH:i:(\d+)', line).group(1)) if
re.search(r'NH:i:(\d+)', line) else 0

# Skip lines where NH (number of reported alignments) is greater than
1
if NH > 1:
    return None

# Determine the strand based on bit flags
if read_reverse_strand:
    strand = '-' if first_in_pair else '+'
else:
    strand = '+' if first_in_pair else '-'

# Extract chromosome and start position, and normalize chromosome
naming
chr = fields[2].replace('chr', '')
if chr == 'X':
    chr = 20
elif chr == 'Y':
    chr = 21
elif not chr.isdigit():
    return None
else:
    chr = int(chr)

start = int(fields[3])
cigar = fields[5]

# Modify the CIGAR string by replacing deletions with matches and
removing insertions and soft clippings
cigar = re.sub(r'\d+I', '', cigar) # Remove insertions
cigar = cigar.replace('D', 'M') # Treat deletions as matches
cigar = re.sub(r'\d+S', '', cigar) # Remove soft clippings

# Consolidate consecutive 'M' operations
pattern = re.compile(r'(\d+)M(\d+)M')
while pattern.search(cigar):
    cigar = pattern.sub(lambda m: f"{int(m.group(1)) +
int(m.group(2))}M", cigar)

return cigar, chr, start, strand

# Read introns and store them in a structured dictionary
with open(intron_file, 'r') as infile:

```

```

for line in infile:
    line = line.strip()
    fields = line.split('\t')
    chr = fields[0].replace('chr', '')
    chr = 20 if chr == 'X' else 21 if chr == 'Y' else int(chr)
    strand = fields[3]
    start, end = int(fields[1]), int(fields[2])
    direction = 'F' if strand == '+' else 'R'
    type_key_start = 'SPOF' if strand == '+' else 'SPOR'
    type_key_end = 'EPOF' if strand == '+' else 'EPOR'

    if chr not in junctions[type_key_start]:
        junctions[type_key_start][chr] = []
        junctions[type_key_end][chr] = []

    junctions[type_key_start][chr].append(start)
    junctions[type_key_end][chr].append(end)
    counts[direction][chr] += 1

T = ["SPOF", "EPOF", "SPOR", "EPOR"]

junction_crossings_counts = {t: {c: [0] * len(junctions[t][c]) for c in
junctions[t]} for t in T}
junction_splicings_counts = {t: {c: [0] * len(junctions[t][c]) for c in
junctions[t]} for t in T}

def do_work(cigar, chr, start, TYPE):
    global junctions, counts, junction_crossings_counts,
junction_splicings_counts, overlap_cutoff
    running_position = start
    return_crosses = ""
    return_splices = ""

    if TYPE in ["SPOF", "EPOF"]:
        dir = "F"
    else:
        dir = "R"

    while True:
        match = re.match(r'^(\d+)([MN])', cigar)
        if not match:
            break
        x = match.group(0)
        N = int(match.group(1))
        cigar = cigar[len(x):]
        S = running_position

        if match.group(2) == 'N':
            running_position += N
        elif match.group(2) == 'M':
            running_position += N
        E = running_position - 1

        current = 1

```

```

current_prev = 0
max = counts[dir][chr]
min = 0
while current_prev != current:
    current_prev = current
    D = start - junctions[TYPE][chr][current]
    if D > 0:
        min = current
        current += (max - current) // 2
    elif D < 0:
        max = current
        current -= (current - min) // 2

current -= 3 # backing up by 3 just to be safe
current_hold = current

if current < 0:
    current = 0
    current_hold = 0

if re.search('M', x):
    # print("chr" + str(chr))
    # print("current" + str(current))
    # print("length " + str(junctions[TYPE][chr][current]))

    while current < len(junctions[TYPE][chr]) and
junctions[TYPE][chr][current] and junctions[TYPE][chr][current] < E:
        junction = junctions[TYPE][chr][current]
        if S < junction < E and (junction - S > overlap_cutoff)
and (E - junction > overlap_cutoff):
            junction_crossings_counts[TYPE][chr][current] += 1
            return_crosses += f"::{TYPE},{chr},{current}"
            current += 1

    current = current_hold # Reset current to its hold value for the
next checks

# Check if 'x' contains 'N' and process accordingly
if re.search('N', x):
    while re.search(r'\S', str(junctions[TYPE][chr][current])) and
junctions[TYPE][chr][current] <= E:
        junction = junctions[TYPE][chr][current]
        if E == junction and TYPE in ["EPOF", "EPOR"]:
            junction_splicings_counts[TYPE][chr][current] += 1
            return_splices += f"::{TYPE},{chr},{current}"
        if S == junction and TYPE in ["SPOF", "SPOR"]:
            junction_splicings_counts[TYPE][chr][current] += 1
            return_splices += f"::{TYPE},{chr},{current}"
        current += 1

    return return_crosses, return_splices

def fix_dups(hits1, hits2, kind):
    global junction_crossings_counts, junction_splicings_counts

```

```

# Append delimiter to hits1 for easy matching
hits1 = hits1 + "::::"

# Remove leading delimiter from hits2 if present
if hits2.startswith("::::"):
    hits2 = hits2[3:]

# Split hits2 by delimiter to get individual records
hits2_list = hits2.split("::::")

# Iterate through the split hits2 list
for hit in hits2_list:
    if not hit.strip(): # Skip if the entry is empty or whitespace
        continue

    # Construct the string to check for presence in hits1
    hit_str = "::::" + hit + "::::"

    # Check if this modified hit is in the hits1 string
    if hit_str in hits1:
        # Split the hit into its components
        details = hit.split(',')
        TYPE, chr, current = details[0], details[1], int(details[2])

        # Adjust counts based on kind ('C' for crossings, 'S' for
splicings)
        if kind == "C":
            junction_crossings_counts[TYPE][chr][current] -= 1
        elif kind == "S":
            junction_splicings_counts[TYPE][chr][current] -= 1

# Process SAM file to count crossings and splicings
with open(samfile, 'r') as sam:
    count = 0
    for line in sam:
        count += 1
        if count % 1000000 == 0:
            print(f"Processed {count} lines", file=sys.stderr)
        if line.startswith('@'):
            continue
        result1 = prepare(line.strip())
        if result1 is None:
            continue
        cigar1, chr1, start1, strand1 = result1
        line = next(sam).strip()
        result2 = prepare(line)
        if result2 is None:
            continue
        cigar2, chr2, start2, strand2 = result2

        if (strand1 == '+'):
            crosses1, splices1 = do_work(cigar1, chr1, start1, "SPOF")
            crosses2, splices2 = do_work(cigar2, chr2, start2, "SPOF")
            if crosses1 and crosses2:

```

```

        fix_dups(crosses1, crosses2, "C")
    if splices1 and splices2:
        fix_dups(splices1, splices2, "S")

    crosses1, splices1 = do_work(cigar1, chr1, start1, "EPOF")
    crosses2, splices2 = do_work(cigar2, chr2, start2, "EPOF")
    if crosses1 and crosses2:
        fix_dups(crosses1, crosses2, "C")
    if splices1 and splices2:
        fix_dups(splices1, splices2, "S")
else:
    crosses1, splices1 = do_work(cigar1, chr1, start1, "SPOR")
    crosses2, splices2 = do_work(cigar2, chr2, start2, "SPOR")
    if crosses1 and crosses2:
        fix_dups(crosses1, crosses2, "C")
    if splices1 and splices2:
        fix_dups(splices1, splices2, "S")

    crosses1, splices1 = do_work(cigar1, chr1, start1, "EPOR")
    crosses2, splices2 = do_work(cigar2, chr2, start2, "EPOR")
    if crosses1 and crosses2:
        fix_dups(crosses1, crosses2, "C")
    if splices1 and splices2:
        fix_dups(splices1, splices2, "S")

# Output results to files
with open(crossings_outfile, 'w') as out_file:
    for j in range(4):
        strand = "+" if j in (0, 1) else "-"
        dir = "F" if j in (0, 1) else "R"
        side = "L" if j in (0, 2) else "R"
        for c in range(1, 22):
            for i in range(counts[dir][c]):
                # Check if the count is greater than zero instead of using
                # .strip() which is for strings
                if junction_crossings_counts[T[j]][c][i] > 0:

out_file.write(f"chr{c}\t{junctions[T[j]][c][i]}\t{strand}\t{side}\t{junct
ion_crossings_counts[T[j]][c][i]}\n")

# Open file to output the splicings
with open(splicings_outfile, 'w') as out_file:
    for j in range(4):
        strand = "+" if j in (0, 1) else "-"
        dir = "F" if j in (0, 1) else "R"
        side = "L" if j in (0, 2) else "R"
        for c in range(1, 22): # assuming chromosome counts from 1 to 21
            for i in range(counts[dir][c]):
                # Check if the count is greater than zero instead of using
                # .strip() which is for strings
                if junction_splicings_counts[T[j]][c][i] > 0:

out_file.write(f"chr{c}\t{junctions[T[j]][c][i]}\t{strand}\t{side}\t{junct
ion_splicings_counts[T[j]][c][i]}\n")

```

Calculate average coverage for each introns found:

```
import sys
import subprocess
import os

# Assigning command line arguments to variables
intron_file = sys.argv[1]
cov_fwd = sys.argv[2]
cov_rev = sys.argv[3]
outfile = sys.argv[4]

def do_work(type):
    if type == "fwd":
        temp_file = "temp.1"
        cov_file = cov_fwd
    elif type == "rev":
        temp_file = "temp.2"
        cov_file = cov_rev

    # Checking for header in the coverage file
    check_header = int(subprocess.getoutput(f"head -1 {cov_file} | grep -c
track"))
    with open(intron_file, 'r') as intron_fh, open(cov_file, 'r') as
cov_fh, open(temp_file, 'w') as out_fh:
        if check_header == 1:
            next(cov_fh) # Skip header if present

        for i in range(1, 22):
            if i == 20:
                j = "X"
                k = 123456789
            elif i == 21:
                j = "Y"
                k = 987654321
            else:
                j = str(i)
                k = i

        print(f"working on chromosome {j}\n")

    # Reset intron file handle
    intron_fh.seek(0)
    skip = False
    for intron in intron_fh:
        parts = intron.strip().split()
        chr_intron = parts[0].lstrip('chr')
        if chr_intron == "X":
            chr_intron = 123456789
        elif chr_intron == "Y":
            chr_intron = 987654321
```

```

        if int(chr_intron) != k:
            continue
        break
    else:
        print(f"No introns on chromosome {j}\n")
        continue # Skip to the next chromosome if no intron found

# Cue up coverage file to this chromosome
cov_fh.seek(0)
if check_header == 1:
    next(cov_fh) # Skip header if present
for cov in cov_fh:
    parts = cov.strip().split()
    chr_cov = parts[0].lstrip('chr')
    if chr_cov == "X":
        chr_cov = 123456789
    elif chr_cov == "Y":
        chr_cov = 987654321
    if int(chr_cov) != k:
        continue
    break
else:
    print(f"No coverage on chromosome {j}\n")
    continue # Skip to the next chromosome if no coverage
found

# Process introns and coverages
intron_fh.seek(0) # Reset intron file handle
for intron in intron_fh:
    parts = intron.strip().split()
    chr_intron, start, end, strand, gid =
parts[0].lstrip('chr'), int(parts[1]), int(parts[2]), parts[3], parts[4]
    if chr_intron == "X":
        chr_intron = 123456789
    elif chr_intron == "Y":
        chr_intron = 987654321
    if int(chr_intron) != k:
        continue

    length = end - start + 1
    total = 0

    # Process coverage within intron boundaries
    for cov in cov_fh:
        parts = cov.strip().split()
        chr_cov, start_cov, end_cov, coverage =
parts[0].lstrip('chr'), int(parts[1]), int(parts[2]), float(parts[3])
        if int(chr_cov) != k:
            break # Exit loop if coverage file moves to next
chromosome

        if start_cov > end or end_cov < start:
            continue # Skip coverage that does not overlap

```

```

        overlap_start = max(start, start_cov)
        overlap_end = min(end, end_cov)
        if overlap_start <= overlap_end:
            total += (overlap_end - overlap_start + 1) *
coverage

        # Calculate average coverage and write to output
        if length > 0:
            ave_cov = total / length

out_fh.write(f"{gid}\t{chr_intron}\t{start}\t{end}\t{strand}\t{total}\t{length}\t{ave_cov}\n")

        cov_fh.seek(0) # Reset coverage file handle for next
intron

def merge_files(file1, file2, outfile):
    keep = {}
    # Read file1 and file2 into keep dictionary
    for temp_file in [file1, file2]:
        with open(temp_file, 'r') as f:
            for line in f:
                line = line.strip()
                key = '\t'.join(line.split()[:5])
                keep[key] = line

    # Write merged results to outfile
    with open(outfile, 'w') as out:
        for key in sorted(keep.keys()):
            out.write(keep[key] + '\n')

def sort_and_finalize(temp_file, final_outfile):
    # Sort temp file and write to final outfile
    sorted_file = f"{temp_file}.sorted"
    subprocess.run(f"sort -k2,2 -k3,3 -k4,4 -n {temp_file} >
{sorted_file}", shell=True)
    with open(sorted_file, 'r') as infile, open(final_outfile, 'w') as
final_out:

        final_out.write("Gene\tchr\tintron_start\tintron_end\tstrand\ttotal_count\t
tintron_length\tavecov\n")
        for line in infile:
            line = line.strip().replace('\t123456789\t',
'\tX\t').replace('\t987654321\t', '\tY\t')
            final_out.write(line + '\n')

# Process introns on forward and reverse strands
print("\nWorking on FORWARD strand introns\n")
print("-----\n")
do_work("fwd")

print("\nWorking on REVERSE strand introns\n")
print("-----\n")
do_work("rev")

```

```
# Merge forward and reverse strand results
print("\nMerging Forward and Reverse\n")
print("-----\n\n")
merge_files("temp.1", "temp.2", "temp.3")

# Sort the merged file and write the final output
sort_and_finalize("temp.3", outfile)
print("DONE!\n\n")
```

References:

Berget, S. M., Moore, C., & Sharp, P. A. (1977). Spliced segments at the 5' terminus of adenovirus 2 late mRNA. *Proceedings of the National Academy of Sciences*, 74(8), 3171-3175.

Lejeune, F., & Maquat, L. E. (2005). Mechanistic links between nonsense-mediated mRNA decay and pre-mRNA splicing in mammalian cells. *Current Opinion in Cell Biology*, 17(3), 309-315.

Mortazavi, A., Williams, B. A., McCue, K., Schaeffer, L., & Wold, B. (2008). Mapping and quantifying mammalian transcriptomes by RNA-Seq. *Nature Methods*, 5(7), 621-628.

Braunschweig, U., Barbosa-Morais, N. L., Pan, Q., Nachman, E. N., Alipanahi, B., Gonatopoulos-Pournatzis, T., Frey, B., Irimia, M., & Blencowe, B. J. (2014). Widespread intron retention in mammals functionally tunes transcriptomes. *Genome Research*, 24(11), 1774-1786.

Ji, Z., Song, R., Regev, A., & Struhl, K. (2015). Many lncRNAs, 5'UTRs, and pseudogenes are translated and some are likely to express functional proteins. *eLife*, 4, e08890.

Ozsolak, F., & Milos, P. M. (2011). RNA sequencing: advances, challenges and opportunities. *Nature Reviews Genetics*, 12(2), 87-98.

Illumina. (n.d.). CAGE-Seq. Illumina. <https://www.illumina.com/science/sequencing-method-explorer/kits-and-arrays/cage-seq.html>

Acknowledgements: We thank Drs. Anthony Mrcela, Nicholas Lahens and Thomas Brooks for copious support, suggestions, help with data handling and system administration.